

What is your role?

Today my position here is supervisor, so I have a role more in leadership, but I still work a lot in development, architecture definition, whatever is needed there, we participate. But the role is more focused on leadership itself.

And how long have you been working with software development?

I have been working since 2014. It has been 12 years.

And which AI tools do you use in your work for this context of software development?

Yeah, today I use Cloud a lot, I think it is pretty standard in the industry now. Then there are other things that I use more to help, there are some plugins, I do not know if I need to mention each one, but there are some more focused plugins, for example, on using fewer tokens, some things more related to optimization for using Cloud, it does not give so much, let us say, it gives a little more context there to the AI, so, the bulk of it is Cloud, but there are several tools there to optimize it a little more. But in general it is even hard to mention a tool, because I am testing a lot of things, because every day something new appears, nowadays this is really crazy.

And how often do you use these tools?

Look, from the beginning of the year until now, almost 100% of the time, so, practically, I do not know, it must have been about 3 or 4 months that I have not actually written a line of code. Sometimes it is just one thing or another there, but it is very detailed, like that.

And in which types of activity do you usually use AI?

Ah, development. I am using it for a lot of things, both tasks and even tasks less related, like bug resolution. Sometimes I take some tools there that we have for observation. I take logs, to take a look. So, like, the development flow, bug solving, code review, in general, I have used it a lot. And the management part there, more to solve some bureaucratic and more annoying things. Sometimes there are some spreadsheets, some things that we have to fill out there, AI also helps a lot, which was something very manual.

And can you describe a recent case in which you used AI to perform a development activity?

I am going to describe here a POC, which I think was an interesting example, because we have a tool today for content management, and this tool is an open-source tool. So, we use it, but we do self-hosting of it there for us, right, to manage, but it is very bad to maintain in general. And then I made a POC in our own repository to try to produce things of this size. The first thing I did was take AI, give it the documentation of the open-source tool, ask it to take the context there and understand what the main functionalities are, raise a document with that, then I took it and connected it there with what we have from the actual implementation to see what we use. And then it had the context of what the tools it has are and how much we actually use, because there is much more than what we actually need. And then after that I asked it to elaborate a plan for what it would be like to do this in the back office. It made the whole plan there, I kept validating what it was doing and then I actually asked it to implement. More specifically, we currently use a little of spec kit for some things. I tried to do an experiment using open spec, which is a slightly lighter version of it in terms of the artifacts it generates there, and this flow was really cool, it was possible to generate little documentation, let us say, and a lot of code, because the tool in general has a lot of little things there. It is a lot of CRUD, but it is a lot of types of CRUD, you know? For content management.

And currently do you use this lighter version of spec kit or the full spec kit in the complete process?

Yeah, I am using this lighter version to experiment. We are even doing, today I am responsible for two teams, in one of them we started experimenting with open spec, I am not sure, let me confirm here. That is right, open spec. We started experimenting with this version precisely because I

worked the context, and because we have a lot of artifacts, a lot of text, sometimes the gain there is not that big, because you have to validate and see a lot of things, you know? And then in the end what you would gain in performance, you were reading a document, so it kind of becomes the same thing.

And in general, this open spec, how many artifacts more or less does it generate in the process for you to do an activity like this, for example?

It depends on the activity, it is hard to say the exact number, but much fewer, you know? Much fewer.

But what do you provide as the main input for it? Is there a PRD there?

Today we are working with the PRD scheme. And basically, our workflow is very focused on PRD, Tech spec, breaking stacks, but, at least, we have used the PRD when it is usually made available by the product team, but when it is by engineering itself, right, it is something that in this POC case, it depends a lot on the feeling of the person doing it there, there is no specific artifact, you know?

I remember a colleague commenting that this initial phase of explaining, writing a good PRD is essential for you to have a good result.

Yeah, I was going to comment, I do not know if eventually you will ask, but this has actually been the biggest bottleneck today in this issue of a good PRD, because product is still adapting a lot to this new type of reality, let us say, and to us being able to generate things from it, you know? But why?

It is because I think the main problem is in size, maybe a little, sometimes they pull very large initiatives with a lot of business rules and it is bad for it to reach us and for us to be able to replicate it or for it not to conflict, because a lot is missing. It still needs, I believe, to reach this middle ground of them actually understanding how to break down deliverable features that work to reach us and for us to develop, because things are still coming very, very large, you know?

Yeah, then a more technical vision really helps in this sense of knowing how to break down these features?

Exactly. And they are also experimenting with a lot of things, including using Copilot connected to the repos so that when they are creating the PRD they really have the context, because there is a lot of business rule itself that is not documented, it is in code. So even for them to get these inputs, but sometimes exactly that happens. It is so much context, so many repos, so much that we use that sometimes it hallucinates completely there.

And in general, when you are programming, you need to make a change there in the code, do you pass a prompt to the LLM, to the AI, to adjust it, or can you go directly to the code?

Look, today I have been doing it through prompt, when I need to adjust something, but it depends a lot. I will give two specific examples. There was an internal implementation that I made where, because I had elaborated the architecture very well, it became very easy to understand what was happening and eventually if I needed to change something it was fine.

But there was another implementation that we did, it was even during, some time ago, when there was kind of an internal hackathon to encourage people to use AI and so on, and the idea was really to develop faster, so it was full vibe code and, like, if I needed to make any change there, I think I would suffer, from what I myself had done. So, like, I felt at least that for things that are better planned, it is okay, if you need to change something, but if it is something 100% in the hands of AI and without any definition, no guardrails, nothing very specific, then it turns into chaos that over time is probably hard to solve.

I wanted to know, for you, what changed with software development with AI, how did it affect your professional life?

Yeah, so, I think, let us go. First, positively, performance increased a lot, because, like it or not, a lot of what we do in development is sometimes just repetitive work. So, you know exactly what needs to be done, but you need to do it, test it, verify it. Anyway, AI gave a real boost to these things. I think also the issue of the ability to experiment with things also increases a lot, because today, if you need to make a proof of concept or something, many times you can just specify it, leave the agents working and go make a coffee and come back and see whether that makes sense or not. That is also very cool. Now, I think negatively, maybe, a little of the so-called FOMO, I think it became a very evident thing, every hour there is someone trying a solution, every hour something better than the other, every hour a model appears and, like, it becomes a craziness in which sometimes it is hard for you to work because you are spending the day changing tools, because people already said that something is much better. So, like, that is one point. The other is also precisely because we are in this transition. There is a lot that is experimental in terms of process, of how the day-to-day functioning works there. Sometimes you pull a task, ah, how am I going to handle this here? Ah, no, this here is an interface that I have to see. I will see, ah, no, I will integrate Figma's MCP and I will ask it to generate the code. Okay. But it will not look good and I need to make a skill. Then you make one. So, the day-to-day process of doing certain things changed, because we are no longer thinking so much about solving a problem for a person, but rather, okay, how do we make the AI solve this type of problem and solve it for everyone, you know? So, it creates a little, sometimes even mental fatigue there, it is a lot of new things at the same time.

I think everyone in the technology area is used to fast change, but AI raised that times 10. I think it is also a lot of adaptation, at least I believe it will be a very chaotic period for a few years, but at some point things will become calmer, you know?

And how was your process of starting to use AI? What difficulties did you encounter in this process of starting to use it?

Look, I am going to break it into two milestones here, because, like, I informed you that I am a physicist by training. And at the time I was in college, my research work was on artificial intelligence focused on autonomous vehicles. So, I worked there on developing models so that a vehicle could basically drive itself. Of course, all in a college context, given the resources we had, nothing close to what we have today, and that was many, many years ago, around 2012, 2013, so at that time what existed in AI, conceptually the issue has not changed that much, but the tooling that existed was, I do not know, 1% of what there is today. So, I always had a good relationship, let us say, but in terms of entering the market, right, of starting to actually use it, I believe I started taking the first steps basically when GPT started to become famous. I do not know if it was 2022, I do not remember the time. When GPT started to become more popular and everyone started talking about AI. But still for very basic, day-to-day things, like, I have a function, I do not know, instead of going to Google and searching, I will ask GPT here to generate it. But really strongly, I believe it was more last year, when at least internally at work we started talking more about it, I started searching more and experimenting more and using it more. So, it must have been, I do not know, about 6, 7, 8 months maybe.

And did you find this change difficult or smooth?

Ah, I did not find it that difficult, but I also did not find it smooth. I think the volume made it much more complicated than the complexity, because the concepts, like it or not, end up being the same, right? When people say: "Ah, they start saying: Ah, MCP, MCP," it is just a communication standard, you know? Then, ah, model X is better for this, model Y is better for that. Ah, okay. Then you find a ranking there that tells you which model is best for each thing. So, since a lot of things started to appear, I think the difficulty is there, but the concept issue, understanding things, has been relatively okay. "Ah, what is a RAG? Ah, RAG is only the information that the AI has access to, you know?". And so on.

And how do you feel using AI today? Do you prefer now or before?

Difficult, that question is good. I really liked coding, it was always something I always enjoyed a lot. So, giving up actually making the solution there is bad, but on the other hand, the tool itself helps me improve a lot of things too, because sometimes, like, when I do not know, you were going to make a solution for something, you elaborated, thought, discussed with someone, searched on Google, I do not know, ah, there is pattern x, y, z that solves this type of problem. It was a very, very large job. Nowadays you exchange 40 minutes of ideas there with Cloud and you arrive at a solution that makes a lot of sense. So, I have enjoyed it, I still like writing code, making solutions, but then I have been leaving that for hobbies, for more specific things, like Arduino, things more like that, that we do for fun, right?

You commented that when you define an architecture well, the code comes out better. At what stage do you define this architecture? And can you explain more or less how you make this definition?

Yeah, when we are doing it through the PRD, Techspec pattern, we usually do it there in the Techspec stage. And normally, we make a very structured prompt, to define what we want from the architecture, but always discussing with the code. So, usually it is a back-and-forth. We raise there, I do not know, an example of something that happened recently, we have a main screen, which is the home screen, and we have a lot of things integrated there and today it has a very specific architecture of how it works. I took the current architecture, threw it to it, said, man, it works like this, this, this. And maybe I wanted something, I do not know, more along the line of you having a port without adapters here. How could we do it? Look, you can do this here, this here, this here. And we keep discussing, keep discussing and in the end, the ideal is to generate a document with the expectations I have. And then after that I ask it, okay, so, since we are going to follow this, this, this, make the development plan here based on this PRD that we have here with these specifications, to change this part of the code following this architecture that was defined. So, normally it is more or less that.

So, it makes a mix there of the PRD, your architectural decision there, and then from that you generate the Techspec, is that it?

Exactly.

And are there situations in which AI does not help as expected?

Only if the model is bad. So, like, in general, as expected, I have never gone through that. Every time it was very good, given the expectation. Then I do not know if my expectation was also self-sufficient. But it was always smooth.

But are there activities where, like, for this activity it is better, but for this one it is not so good?

So, at the beginning we were talking a little about the issue of interface development, because for, I do not know, an endpoint, anything on the side of a service layer, it is always more straightforward. Because, like, it is literally the code it needs to write and that is it. Now, if I take an interface and there is an animation, it is no longer as performant as for developing, I do not know, an integration with a database. But with the tools we have been experimenting with, it is improving. For example, at least the static interface, we are moving well. So, we connect there to Figma and we can already generate code in a very faithful, very expected way. For animation, I do not know, we are still in this process, but for now, it still does not seem to be that good, but it may just be our lack of knowledge that we will still discover how to do this better.

What do you think is still missing to improve the use of AI in software development? Do you have any recommendation, any practice?

I think that today, I will answer in the context in which I am working today, but I think a better definition of end-to-end flows is missing. Because today everything is still very much based on experimentation. So, for example, when we, I do not know, generated the Techspec, did the

implementation, one thing that we still do not do, for example, is to have an automated testing flow to ensure that it works. Even if there is, how am I going to be sure that my flow makes sense? Okay, I can review these specs there, the generated things and have some guarantee. But, like, the flow is still not 100% integrated and I think what needs to happen is that, this flow has to be 100% integrated.